

CSE310 Project 3: Single-pair and Single-source Shortest Paths

Professor Guoliang Xue

11/04/2025: PJ03 released.

11/11/2025: Checkpoint 1 due.

11/18/2025: Checkpoint 2 due.

11/25/2025: PJ03 due without late penalty.

12/07/2025: PJ03 due with 20-point late penalty.

This should be your individual work. As in the case of your first two projects, it should be written using the standard C++ programming language, and compiled using the g++ compiler on a Linux platform. Your project will be graded on Gradescope. If you compile your project on `general.asu.edu` using the compiler commands provided in the sample `Makefile`, you should expect the same behavior of your project on both `general.asu.edu` and Gradescope. Therefore, you are highly recommended to develop your project on `general.asu.edu`.

In this project, you will implement a simple navigation system using Dijkstra's shortest path algorithms. You need to use adjacency lists as the graph representation. You need to implement a min-heap as the priority queue. You need to implement a stack to trace the computed shortest path. Vertex attributes should be properly stored for efficient computation.

You have three weeks to complete this project. You also have an option to have 12 more days to work on the project with a 20-point late penalty. All dates are set. No extension will be given.

In the first two projects, the checkpoints carry 0 weight. **As an incentive to work on the project early, in this project, Checkpoint 1 and Checkpoint 2 each carries a 5-point extra credit. Specifically, if your submission to Checkpoint 1 (Checkpoint 2) earns C_1 (C_2) points out of the maximum possible 100 points, you will earn an extra credit of $5 \times \frac{C_1}{100} + 5 \times \frac{C_2}{100}$ for the course.**

1 Functionalities of the Project

This project is the essential part of a navigation system. It reads in either a directed graph or an undirected graph G with n vertices and m edges from a file specified by the command-line (with proper arguments). It then takes corresponding actions for given instructions from `stdin`. Besides the `Stop` instruction, valid instructions include

- (i) graph printing instruction (`PrintAdj`)
- (ii) path computation instructions (`SinglePair` and `SingleSource`)
- (iii) length/path printing instructions (`PrintLength` and `PrintPath`)

Both `SinglePair` and `SingleSource` path computations should have worst-case time complexity $O((m + n) \log n)$. Path printing should have worst-case time complexity $O(n)$. Length printing should have worst-case time complexity $O(1)$. Memory should be allocated when needed, and released when it is no longer needed. Memory leaks should be avoided. **Some of the test cases will fail programs that have a time complexity higher than expected and programs that do not do proper memory management.**

2 Modular Design

For modular design, you need to write the following modules:

- a module for various **data structures**, with header file named `data_structures.h`
- a module for **min-heap**, with header file named `heap.h` and implementation file named `heap.cpp`
- a module for **stack**, with header file named `stack.h` and implementation file named `stack.cpp`
- a module for **graph algorithms**, with header file named `graph.h` and implementation file named `graph.cpp`
- a module for **utilities**, with header file named `util.h` and implementation file named `util.cpp`
- and the **main program**, with header file named `main.h` and implementation file named `main.cpp`

Your project should only use basic C++ libraries to compile your implementation to produce the executable PJ3. To ensure that, the following standard **Makefile** will be used to grade your project.

```
EXEC = PJ3
CC = g++
CFLAGS = -c -Wall

$(EXEC) :main.o util.o stack.o heap.o graph.o
        $(CC) -o $(EXEC) main.o util.o heap.o graph.o stack.o

main.o :main.cpp main.h data_structures.h util.h stack.h heap.h graph.h
        $(CC) $(CFLAGS) main.cpp

util.o :util.cpp util.h data_structures.h
        $(CC) $(CFLAGS) util.cpp

stack.o :stack.cpp stack.h data_structures.h
        $(CC) $(CFLAGS) stack.cpp

heap.o :heap.cpp heap.h data_structures.h
        $(CC) $(CFLAGS) heap.cpp

graph.o :graph.cpp graph.h data_structures.h stack.h heap.h
        $(CC) $(CFLAGS) graph.cpp

clean :
        rm *.o $(EXEC)
```

Given that your project will be compile using the above Makefile, you have to name the files exactly as specified.

3 Data Structures

For this project, you need to implement graph representation using adjacency lists, a priority queue (min-heap), a stack, and the graph algorithms related to Dijkstra's shortest path algorithm. This section provides some data structures that may give you a head start with the project. You should not copy and paste these data structures. However, you can write similar ones after studying them. It should be your own work.

3.1 VERTEX

You need to have a data structure for vertices in the graph. Our algorithms also use a color system on vertices. The following is a part of the codes that I wrote. Note that the definition of `COLOR` in the following is NOT provided.

```
typedef struct TAG_VERTEX{
    int      index;
    COLOR     color;
    double    key;
    int      pi;
    int      position;
}VERTEX;
typedef VERTEX *pVERTEX;
```

Here the field `index` is the index of the corresponding vertex. It does not change. The fields `key` is the length of the *current best path from the source* computed. The field `position` is used to record the **position of the vertex in the min-heap array**. You will find this very handy when `DecreaseKey` operations are needed. I use the `key` field (rather than the `d` field) to reuse my codes for the min-heap.

Here is a sample usage of the data type `VERTEX`.

```
pVERTEX *V;
// V is a pointer to pointer to VERTEX

V = (pVERTEX *) calloc(n+1, sizeof(pVERTEX));
// Now you can make reference to V[1], but not to V[1]->index

for (int i=1; i<=n; i++){
    V[i] = (VERTEX *) calloc(1, sizeof(VERTEX));
    // Now you can make reference to V[1]->index
}
```

Please note that illegal memory access is the main cause of segmentation fault.

3.2 EDGE

You need to have a data structure for nodes on the adjacency lists of the graph. I wrote the following in my implementation.

```
typedef struct TAG_NODE{
    int      index;
    int      u;
    int      v;
    double   w;
    TAG_NODE *next;
}NODE;
typedef NODE *pNODE;
```

Here the field `index` is the index of the edge. The field `w` is the weight of the corresponding edge. The fields `u` and `v` correspond to the two end vertices of this edge. The usage of the `NODE` data type is similar to that of `VERTEX`.

3.3 MIN-HEAP

In Project 2, you have implemented the min-heap data structure, where the `HEAP` data structure contains the fields `capacity` (of type `int`), `size` (of type `int`), and `H` (of type `int *`). In this project, you need to modify it so that `H` is of type `ELEMENT **`. The data structure `ELEMENT` contains only one required field: `key` (of type `double`).

To partially re-use your implementation of Project 2, you can use the following:

```
typedef VERTEX    ELEMENT;
typedef ELEMENT *pELEMENT;
```

Because `VERTEX` has a field named `key` (of type `double`), your earlier implementations of min-heap should work as usual. Note that you can make reference to the four other fields defined in `VERTEX`, and in `ELEMENT`.

In this project, the elements of the heap array should be pointers to objects of type `VERTEX`. To reuse codes with minimal modification, I used the following in my implementation.

```
typedef VERTEX    ELEMENT;
typedef ELEMENT *pELEMENT;
typedef struct TAG_HEAP{
    int      capacity;
    int      size;
    pELEMENT *H;
}HEAP;
typedef HEAP *pHEAP;
```

3.4 STACK

In order to print out the computed path from a source node s to a destination node t , we start from the destination node to trace out the path using the predecessor field. This will trace out the path in reverse order. In order to print the path in correct order, we need to use a stack. The elements of the stack should contain information (or pointers to information) about the corresponding vertices on the path.

4 Shortest Path Algorithm

For improved efficiency, you should implement the following version of Dijkstra's shortest s-t path algorithm.

```
Dijkstra-ST(G, w, s, t)
01: FOR v=1 TO n {
02:     V[v]->index    = v;
03:     V[v]->key       = infinity;
04:     V[v]->pi        = NULL;
05:     V[v]->color     = WHITE;
06:     V[v]->position = 0;
07: }
08: Create empty min-heap pHeap;
09: Set the key of s to 0, and the color of s to GRAY;
10: Insert s into pHeap;
11: WHILE pHeap->size > 0 {
12:     u = ExtractMin(pHeap);
13:     IF (u == t) THEN STOP;
14:     FOR each v on ADJ[u] {
15:         IF v is WHITE {
16:             Set the color of v to GRAY;
16:             Update v's key, pi; // Relax(u, v, w);
17:             Insert v into pHeap;
18:         }ELSE IF v is GRAY {
19:             Relax(u, v, w);
20:             Call DecreaseKey if nexessary; // Using the position field of v
21:         }
22:     }
23:     Color u BLACK;
24: }
```

For single-pair shortest path, both s and t are in $\{1, 2, \dots, n\}$. For single-source shortest paths, s is in $\{1, 2, \dots, n\}$, t is set to -1 .

5 Valid Executions

A valid execution of your project has the following form:

```
./PJ3 <InputFile> <GraphType> <Flag>
```

where

- PJ3 is the executable file of your project,
- <InputFile> should be the exact name of the input file,
- <GraphType> should be substituted by either `DirectedGraph` or `UndirectedGraph`,
- <Flag> is either 1 or 0.

Your program should check whether the execution is valid. If the execution is not valid, your program should print out the following message to `stderr` and stop.

```
Usage: ./PJ3 <InputFile> <GraphType> <Flag>
```

Note that your program should not crash when the execution is not valid.

6 Flow of the Project

6.1 Read in the Graph and Build the Adjacency Lists

Upon a valid execution, your program should open the input file (specified by `argv[1]`) and read in the graph. The format of the input file is the following. The first line contains two positive integers, representing the number of vertices n and number of edges m , respectively. Each of the next m lines has the following format:

```
edgeIndex u v w(u, v)
```

where u is the start vertex of edge (u, v) , v is the end vertex of edge (u, v) , $w(u, v)$ is the weight of edge (u, v) , and *edgeIndex* is the index of edge (u, v) . Your program should read in *edgeIndex*, *u*, and *v* as `int`, and read in *w(u, v)* as `double`. *For undirected graphs, each edge still has two vertices but the order is not important.*

After knowing the values of n and m , your program should dynamically allocate memory for an array of n pointers to objects of type `VERTEX`. Let us call this array `V`. Then for each $i = 1, 2, \dots, n$, `V[i]` is a pointer to an object for vertex i . Your program should dynamically allocate memory for an array of n adjacency lists. Let us call this array `ADJ`. Then for each $i = 1, 2, \dots, n$, `ADJ[i]` is a pointer to `NODE`. Note that `NODE` has a field `next`. Therefore `ADJ[i]` is a pointer to the adjacency list of vertex i .

While reading the graph from `argv[1]` edge by edge, the adjacency lists are populated accordingly. Note that dynamic memory allocation is needed for each node (corresponding to an edge) on the adjacency lists.

Let (u, v) be the edge newly read in from the file `argv[1]`. If the graph is directed and `flag` is 1, insert the corresponding node for edge (u, v) at the rear of `ADJ[u]`. If the graph is directed and `flag` is 0, insert the corresponding node for edge (u, v) at the front of `ADJ[u]`. If the graph is undirected and `flag` is 1, insert a corresponding node for edge (u, v) at the rear of `ADJ[u]`, and insert a corresponding node for edge (v, u) at the rear of `ADJ[v]`. If the graph is undirected and `flag` is 0, insert a corresponding node for edge (u, v) at the front of `ADJ[u]`, and insert a corresponding node for edge (v, u) at the front of `ADJ[v]`. After all m edges are read in, your program should close the file `argv[1]`.

6.2 Initialize the Priority Queue (min-heap) and the Stack

Your program should initialize a min-heap of capacity n and a stack of capacity n . Dynamic memory allocations are required for the min-heap and the stack. The heap will be used in the execution of Dijkstra's path finding algorithm. The stack will be used to output a path from a given source vertex to a given destination vertex.

6.3 Loop over the Instructions

Your program should expect the following instructions from `stdin` and act accordingly:

- (a) **Stop**
On reading **Stop**, the program stops.
- (b) **PrintADJ**
On reading the **PrintADJ** instruction, your program should do the following:
 - (b-i) Print the adjacency lists of the input graph to `stdout`. Refer to posted test cases for output format.
 - (b-ii) Wait for the next instruction from `stdin`.
- (c) **SinglePair <source> <destination>**
where `<source>` and `<destination>` are two integers in the set $\{1, 2, \dots, n\}$. **This is one of the two *path computation* instructions.** On reading the **SinglePair** instruction, your program should do the following:
 - (c-i) Apply the variant of Dijkstra's algorithm (as taught in class and stated in the lecture slides) to compute a shortest path from `<source>` to `<destination>`. In particular, for each $i \in \{1, 2, \dots, n\}$, the algorithm should compute the final values of $V[i] \rightarrow \text{key}$ and $V[i] \rightarrow \pi$. If `<destination>` is reachable from `<source>`, $V[\text{destination}] \rightarrow \text{key}$ is the weight of a shortest path from `<source>` to `<destination>`, and $V[\text{destination}] \rightarrow \pi$ is the predecessor of `<destination>` on the computed shortest path from `<source>` to `<destination>`. If `<destination>` is not reachable from `<source>`, $V[i] \rightarrow \pi$ is `nil`, and $V[i] \rightarrow \text{key}$ is `DBL_MAX` (defined in the header file `<float>`).
 - (c-ii) Wait for the next instruction from `stdin`.

(d) **SingleSource** <source>

where <source> is an integer in the set $\{1, 2, \dots, n\}$. **This is the other *path computation instruction*.** On reading the SingleSource instruction, your program should do the following:

(d-i) Apply Dijkstra's algorithm to compute shortest paths from <source> to all vertices that are reachable from <source>. In particular, for each $i \in \{1, 2, \dots, n\}$, the algorithm should compute the final values of $V[i] \rightarrow \text{key}$ and $V[i] \rightarrow \pi$. If vertex i is reachable from <source>, $V[i] \rightarrow \text{key}$ is the length of a shortest path from <source> to i , and $V[i] \rightarrow \pi$ is the predecessor of i on the computed shortest path from <source> to i . If vertex i is not reachable from <source>, $V[i] \rightarrow \pi$ is nil, and $V[i] \rightarrow \text{key}$ is DBL_MAX.

(d-ii) Wait for the next instruction from `stdin`.

(e) **PrintLength** <s> <t>

where <s> and <t> are two integers in the set $\{1, 2, \dots, n\}$. **This is the only *length printing instruction*.** This instruction is valid if and only if <s> is the same as <source> in the most recent path computation instruction, and <t> is the same as <destination> in case the most recent path computation instruction is a SinglePair instruction.

On reading a valid PrintLength instruction, your program should do the following:

(e-i) If your program has computed a shortest <s> to <t> path in the most recent path computation, print the length of the computed path to `stdout`. Refer to posted test cases for output format.

If your program has not computed a shortest <s> to <t> path in the most recent path computation, print the following to `stdout`:

There is no path from <s> to <t>.

In the above, <s> and <t> should be replaced by their given values. Refer to posted test cases for output format.

(e-ii) Wait for the next instruction from `stdin`.

(f) **PrintPath** <s> <t>

where <s> and <t> are two integers in the set $\{1, 2, \dots, n\}$. **This is the only *path printing instruction*.** This instruction is valid if and only if <s> is the same as <source> in the most recent path computation instruction, and <t> is the same as <destination> in case the most recent path computation instruction is a SinglePair instruction.

On reading a valid PrintPath instruction, your program should do the following:

(f-i) If your program has computed a shortest <s> to <t> path in the most recent path computation, print the computed path to `stdout`. Refer to posted test cases for output format.

If your program has not computed a shortest <s> to <t> path in the most recent path computation, print the following to `stdout`:

There is no path from `<s>` to `<t>`.

In the above, `<s>` and `<t>` should be replaced by their given values. Refer to posted test cases for output format.

(f-ii) Wait for the next instruction from `stdin`.

(g) **Invalid instruction**

On reading an invalid instruction, your program should do the following:

(g-i) Write the following to `stderr`:

`Invalid instruction.`

(g-ii) Wait for the next instruction from `stdin`.

7 Format of the Input File

Refer to Section 6.1 and the posted test cases for the format of the input file. While `w(u, v)` may look like an `int` in the input file, it should be read in as `double`.

8 Format of the Output

Since the autograder on Gradescope uses the Linux `diff` function to compare your output with the expected output, your output has to match the expected output exactly. It is your responsibility to make sure that your output matches the expected output, with the aid of the posted test cases. The following are some general guidelines to help you.

8.1 Output Integers to `stdout`

All integers (to `stdout`) should be printed using the `%d` format. Refer to the posted test cases.

8.2 Output Floating Point Numbers to `stdout`

Edge weights (to `stdout`) should be printed using the `%4.21f` format when printing the adjacency lists (for easy debugging). In all other places, doubles (to `stdout`) should be printed using the `%8.21f` format. Refer to the posted test cases.

8.3 Output Strings (possibly with numerical values) to `stdout`

Refer to the posted test cases for strings printed to `stdout`. Keep in mind the rules in Section 8.1 (for `int`) and Section 8.2 (for `double`).

8.4 Output to `stderr`

While the autograder does not test the messages written to `stderr`, your program should output meaningful messages with sufficient information when a warning or error message should be written to `stderr`.

9 Submission

You should submit your project to Gradescope via the link on Canvas. Submit all header files and implementation files. You should put your name and ASU ID at the top of each of the header files and the implementation files, as a comment.

The instructor and the TAs will offer more help to this project early on, and will not answer emails/questions near the project due date that are clearly in the very early stage of the project. So, please **manage your time, and start working on this project today**.

You should not use Gradescope and the autograder on Gradescope as a debugger. The main purpose of the test cases are for you to get familiar with the output format. It is suggested to use a similar shell script to test your program on `general.asu.edu`. You also need to submit to Gradescope early enough to avoid last-minute surprises.

10 Grading

All programs will be compiled and graded on Gradescope. If your program does not compile and work on Gradescope, you will receive 0 on this project. If your program works well on `general.asu.edu`, there should not be much problems. The maximum possible points for this project is 100. The following shows how you can have points deducted.

1. **Non-working program:** If your program does not compile or does not execute on Gradescope, you will receive a 0 on this project. Do not claim “my program works perfectly on my PC, but I do not know how to use Gradescope.”
2. **Program requires non-standard libraries:** If your program does not compile/working using the compiler and flags as stated in the sample Makefile, 20 points will be marked off.
3. **Posted test cases:** For each of the posted test cases that your program fails, 4 points will be marked off.
4. **Hidden test cases:** For each of the Hidden test cases that your program fails, 4 points will be marked off.

11 Examples and Test Cases

Test cases will be posted on Canvas shortly. The following is the content of the input file `network01.txt`, which is a directed graph with $n = 8$ and $m = 14$.

```
8 14
1 1 2 10
2 1 4 5
3 2 3 1
4 2 4 2
```

```

5 3 5 4
6 4 2 3
7 4 3 9
8 4 5 2
9 5 1 7
10 5 3 6
11 6 3 7
12 6 7 5
13 7 8 3
14 8 6 1

```

This graph is illustrated in Figure 1. You are highly encouraged to debug your program using this directed graph.

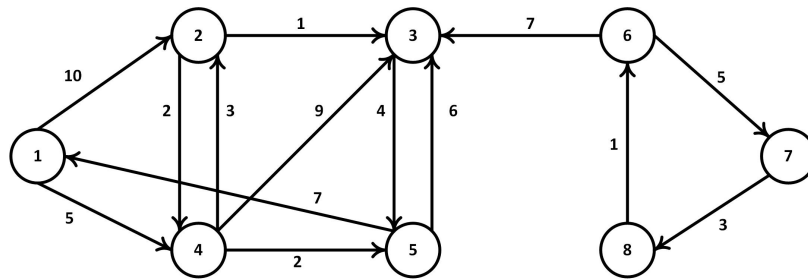


Figure 1: Directed graph G_1 contained in the file `network01.txt`

The following is the content of the input file `network02.txt`, which is an undirected graph with $n = 8$ and $m = 11$.

```

8 11
1 1 2 10
2 1 4 5
3 1 5 7
4 2 3 1
5 2 4 3
6 3 4 9
7 3 5 4
8 3 6 7
9 4 5 2
10 5 6 6
11 7 8 3

```

This graph is illustrated in Figure 2. You are highly encouraged to debug your program using this undirected graph.

Some examples are presented in the following subsections. I used the shell script `run` with the following content

```

#!/bin/bash
./Execution < Instructions > Output.txt

```

as sample executions for given files `Execution` and `Instructions` to generate these examples.

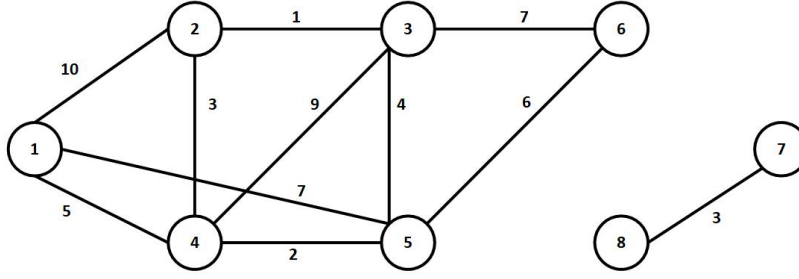


Figure 2: Undirected graph G_2 contained in the file `network02.txt`

11.1 Example 1

The content of `Execution` is

```
#!/bin/bash
./PJ3 network01.txt DirectedGraph 1
```

The content of `Instructions` is

```
PrintAdj
Stop
```

The content of the produced `Output.txt` is

```
Adj[1]:-->[1 2: 10.00]-->[1 4: 5.00]
Adj[2]:-->[2 3: 1.00]-->[2 4: 2.00]
Adj[3]:-->[3 5: 4.00]
Adj[4]:-->[4 2: 3.00]-->[4 3: 9.00]-->[4 5: 2.00]
Adj[5]:-->[5 1: 7.00]-->[5 3: 6.00]
Adj[6]:-->[6 3: 7.00]-->[6 7: 5.00]
Adj[7]:-->[7 8: 3.00]
Adj[8]:-->[8 6: 1.00]
```

Please note that the nodes are inserted **at the rear** of the adjacency list.

11.2 Example 2

The content of `Execution` is

```
#!/bin/bash
./PJ3 network01.txt DirectedGraph 0
```

The content of `Instructions` is

```
PrintAdj
Stop
```

The content of the produced `Output.txt` is

```

ADJ[1]:-->[1 4: 5.00]-->[1 2: 10.00]
ADJ[2]:-->[2 4: 2.00]-->[2 3: 1.00]
ADJ[3]:-->[3 5: 4.00]
ADJ[4]:-->[4 5: 2.00]-->[4 3: 9.00]-->[4 2: 3.00]
ADJ[5]:-->[5 3: 6.00]-->[5 1: 7.00]
ADJ[6]:-->[6 7: 5.00]-->[6 3: 7.00]
ADJ[7]:-->[7 8: 3.00]
ADJ[8]:-->[8 6: 1.00]

```

Please note that the nodes are inserted **at the front** of the adjacency list.

11.3 Example 3

The content of Execution is

```

#!/bin/bash
./PJ3 network02.txt UndirectedGraph 1

```

The content of Instructions is

```

PrintADJ
Stop

```

The content of produced Output.txt is

```

ADJ[1]:-->[1 2: 10.00]-->[1 4: 5.00]-->[1 5: 7.00]
ADJ[2]:-->[2 1: 10.00]-->[2 3: 1.00]-->[2 4: 3.00]
ADJ[3]:-->[3 2: 1.00]-->[3 4: 9.00]-->[3 5: 4.00]-->[3 6: 7.00]
ADJ[4]:-->[4 1: 5.00]-->[4 2: 3.00]-->[4 3: 9.00]-->[4 5: 2.00]
ADJ[5]:-->[5 1: 7.00]-->[5 3: 4.00]-->[5 4: 2.00]-->[5 6: 6.00]
ADJ[6]:-->[6 3: 7.00]-->[6 5: 6.00]
ADJ[7]:-->[7 8: 3.00]
ADJ[8]:-->[8 7: 3.00]

```

11.4 Example 4

The content of Execution is

```

#!/bin/bash
./PJ3 network02.txt UndirectedGraph 0

```

The content of Instructions is

```

PrintADJ
Stop

```

The content of produced Output.txt is

```

ADJ[1]:-->[1 5: 7.00]-->[1 4: 5.00]-->[1 2: 10.00]
ADJ[2]:-->[2 4: 3.00]-->[2 3: 1.00]-->[2 1: 10.00]
ADJ[3]:-->[3 6: 7.00]-->[3 5: 4.00]-->[3 4: 9.00]-->[3 2: 1.00]
ADJ[4]:-->[4 5: 2.00]-->[4 3: 9.00]-->[4 2: 3.00]-->[4 1: 5.00]
ADJ[5]:-->[5 6: 6.00]-->[5 4: 2.00]-->[5 3: 4.00]-->[5 1: 7.00]
ADJ[6]:-->[6 5: 6.00]-->[6 3: 7.00]
ADJ[7]:-->[7 8: 3.00]
ADJ[8]:-->[8 7: 3.00]

```

11.5 Example 5

The content of Execution is

```

#!/bin/bash
./PJ3 network01.txt DirectedGraph 1

```

The content of Instructions is

```

SinglePair 1 3
PrintPath 1 3
Stop

```

The content of the produced Output.txt is

The shortest path from 1 to 3 is:

```
[1:    0.00]-->[4:    5.00]-->[2:    8.00]-->[3:    9.00].
```

11.6 Example 6

The content of Execution is

```

#!/bin/bash
./PJ3 network01.txt DirectedGraph 0

```

The content of Instructions is

```

SinglePair 1 3
PrintPath 1 3
Stop

```

The content of the produced Output.txt is

The shortest path from 1 to 3 is:

```
[1:    0.00]-->[4:    5.00]-->[2:    8.00]-->[3:    9.00].
```

11.7 Example 7

The content of Execution is

```
#!/bin/bash
./PJ3 network01.txt DirectedGraph 0
```

The content of Instructions is

```
SinglePair 1 6
PrintPath 1 6
Stop
```

The content of the produced Output.txt is

There is no path from 1 to 6.

11.8 Example 8

The content of Execution is

```
#!/bin/bash
./PJ3 network02.txt UndirectedGraph 0
```

The content of Instructions is

```
SinglePair 1 3
PrintPath 1 3
Stop
SingleSource 1
SinglePair 1 3
```

The content of the produced Output.txt is

The shortest path from 1 to 3 is:

[1: 0.00]-->[4: 5.00]-->[2: 8.00]-->[3: 9.00].

11.9 Example 9

The content of Execution is

```
#!/bin/bash
./PJ3 network02.txt UndirectedGraph 0
```

The content of Instructions is

```
SinglePair 1 8
PrintPath 1 8
Stop
SingleSource 1
SinglePair 1 3
```

The content of the produced Output.txt is

There is no path from 1 to 8.

11.10 Example 10

The content of Execution is

```
#!/bin/bash
./PJ3 network01.txt DirectedGraph 0
```

The content of Instructions is

```
SingleSource 1
PrintPath 1 5
PrintPath 1 8
Stop
SingleSource 1
SinglePair 1 3
```

The content of the produced Output.txt is

```
The shortest path from 1 to 5 is:
[1:    0.00]-->[4:    5.00]-->[5:    7.00].
There is no path from 1 to 8.
```

11.11 Example 11

The content of Execution is

```
#!/bin/bash
./PJ3 network01.txt DirectedGraph 0
```

The content of Instructions is

```
SingleSource 1
PrintLength 1 5
PrintLength 1 8
Stop
SingleSource 1
SinglePair 1 3
```

The content of the produced Output.txt is

```
The length of the shortest path from 1 to 5 is:    7.00
There is no path from 1 to 8.
```

11.12 Example 12

The content of Execution is

```
#!/bin/bash
./PJ3 network02.txt UndirectedGraph 0
```


The content of Instructions is

```
SingleSource 1
PrintPath 1 5
PrintPath 1 8
Stop
SingleSource 1
SinglePair 1 3
```

The content of the produced Output.txt is

```
The shortest path from 1 to 5 is:
[1:    0.00]-->[5:    7.00].
There is no path from 1 to 8.
```

11.13 Example 13

The content of Execution is

```
#!/bin/bash
./PJ3 network02.txt UndirectedGraph 0
```

The content of Instructions is

```
SingleSource 1
PrintLength 1 5
PrintLength 1 8
Stop
SingleSource 1
SinglePair 1 3
```

The content of the produced Output.txt is

```
The length of the shortest path from 1 to 5 is:    7.00
There is no path from 1 to 8.
```

11.14 Example 14

The content of Execution is

```
#!/bin/bash
./PJ3 network03.txt DirectedGraph 0
```

The content of Instructions is

```
SingleSource 1
PrintPath 1 2
PrintPath 1 3
PrintPath 1 4
```

```
PrintPath 1 5
PrintPath 1 6
PrintPath 1 7
PrintPath 1 8
PrintPath 1 9
Stop
```

The content of the produced `Output.txt` is

```
The shortest path from 1 to 2 is:
[1: 0.00]-->[2: 9.93].
The shortest path from 1 to 3 is:
[1: 0.00]-->[3: 1.36].
The shortest path from 1 to 4 is:
[1: 0.00]-->[3: 1.36]-->[4: 1.76].
The shortest path from 1 to 5 is:
[1: 0.00]-->[3: 1.36]-->[4: 1.76]-->[5: 4.18].
The shortest path from 1 to 6 is:
[1: 0.00]-->[3: 1.36]-->[4: 1.76]-->[5: 4.18]-->[6: 9.10].
The shortest path from 1 to 7 is:
[1: 0.00]-->[3: 1.36]-->[4: 1.76]-->[5: 4.18]-->[6: 9.10]-->[7: 23.51].
The shortest path from 1 to 8 is:
[1: 0.00]-->[2: 9.93]-->[8: 12.69].
The shortest path from 1 to 9 is:
[1: 0.00]-->[2: 9.93]-->[8: 12.69]-->[9: 19.88].
```

11.15 Example 15

The content of `Execution` is

```
#!/bin/bash
./PJ3 network03.txt DirectedGraph 0
```

The content of `Instructions` is

```
SingleSource 1000
PrintPath 1000 1010
Stop
```

The content of produced `Output.txt` is

```
The shortest path from 1000 to 1010 is:
[1000: 0.00]-->[1004: 1.24]-->[1006: 3.54]-->[1010: 7.15].
```

11.16 Example 16

The content of Execution is

```
#!/bin/bash
./PJ3 network01.txt DirectedGraph 2
```

The content of Instructions is

```
PrintADJ
Stop
```

The file Output.txt is empty. The following message is written to stderr.

```
Usage: ./PJ3 <InputFile> <GraphType> <Flag>
```

11.17 Example 17

The content of Execution is

```
#!/bin/bash
./PJ3 network01.txt DirectedGraph 0
```

The content of Instructions is

```
PrintPath 1 3
Stop
```

The file Output.txt is empty. The following is writrten to stderr.

```
Error: Invalid instruction.
```

11.18 Example 18

The content of Execution is

```
#!/bin/bash
./PJ3 network01.txt DirectedGraph 0
```

The content of Instructions is

```
SingleSource 1
PrintPath 1 2
PrintPath 2 3
PrintPath 1 8
SingleSource 2
PrintPath 1 2
PrintPath 2 5
Stop
```

The content of produced `Output.txt` is

The shortest path from 1 to 2 is:

[1: 0.00]-->[4: 5.00]-->[2: 8.00].

There is no path from 1 to 8.

The shortest path from 2 to 5 is:

[2: 0.00]-->[4: 2.00]-->[5: 4.00].

12 Suggested Schedule

The workload for this project is not light. It is recommended that you start working on it immediately.

I suggest that you try the following schedule, starting immediately.

1. **Day 1:** Get the data structures implemented and compiled on `general.asu.edu`. You can have each of the functions having an empty shell for now (just to have the correct syntax).
2. **Day 2:** Read in the graph and build the adjacency lists for directed graphs, with `flag=0`, using `network01.txt`. Make sure it compiles and works correctly on `general.asu.edu`. You can use Example 1 as an aid. **Submit to Gradescope Checkpoint 1 and see how many test cases your program passes.**
3. **Day 3:** Read in the graph and build the adjacency lists for directed graphs, with `flag=1`, using `network01.txt`. Make sure it compiles and works correctly on `general.asu.edu`. You can use Example 2 as an aid. **Submit to Gradescope Checkpoint 1 and see how many test cases your program passes.**
4. **Day 4:** Read in the graph and build the adjacency lists for undirected graphs, with `flag=0` and 1, using `network02.txt`. Make sure it compiles and works correctly on `general.asu.edu`. You can use Examples 3 and 4 as an aid. **Submit to Gradescope Checkpoint 1 and see how many test cases your program passes.**
5. **Day 5:** Read in all valid instructions, and print the instructions to `stderr`. Make sure it compiles and works correctly on `general.asu.edu`. You should be able to get this done easily after the first project. **Submit to Gradescope Checkpoint 1 and see how many test cases your program passes.**
6. **Day 6:** Write the function for `SinglePair` and `PrintPath`. Make sure it compiles and works correctly on `general.asu.edu`. **Submit to Gradescope Checkpoint 1 and see how many test cases your program passes.**
7. **Day 7:** Write the function for `SingleSource` and `PrintLength`. Make sure it compiles and works correctly on `general.asu.edu`. **Submit to Gradescope Checkpoint 1 and see how many test cases your program passes.**

8. **Day 8:** Work on all possible cases you can think of (including invalid instructions). Work on output format (use the `diff` utility to compare files). **Submit to Gradescope Checkpoint 1 and see how many test cases your program passes.**
9. **Day 9 and More:** In case your program does not pass all the test cases, try to figure out the reason, and fix it.

If you follow the above schedule, you should be able to complete this project using half of the time given. If you get one day's work in two days, you should be on target to get a perfect score. If you cannot keep up with the proposed schedule, you need to work harder. Please note that the first few test cases are easy, but some of the later ones are much harder. Start early and manage your time wisely.

Good Luck!